


```
showLevelIntro(cb) {  
    const cfg = this.levelConfigs[this.level - 1];  
    const div = document.createElement('div');  
    div.className
```



```
if (this.cards[a] === this.cards[b]) {
```

```
  this.matched.push(a, b);
```

```
const cards = this.container.querySelectorAll('.mem
```



```
destroy() { clearInterval(this.timer); }); // == whack-a-Mole (js/games/whack.js) == /** * Whack-a-Mole (bSw0Y )*-@5uQb0fQ6j2Q77
```



```
Puzzle Games Collection V3.2
    const cfg = this.levelConfigs[this.level - 1];    const div = document.createElement('div');    div.className = 'level-intro';    d
```



```
}); }, startSpawner() { const spawn = () => { if (!this.running) return; const hideIdx = Math.floor(Math.random() *
```



```
this.timer = setInterval(() => {      this.timeLeft--;      this.updateStats();      if (this.timeLeft <= 0) this.gameOver(false);
```



```
const WordleGame = {
  words: ['apple', 'banana', 'cherry', 'date', 'elderberry', 'fig', 'grape', 'honeydew', 'kiwi', 'lemon', 'mango', 'orange', 'peach', 'pineapple', 'plum', 'raspberry', 'strawberry', 'tangerine', 'watermelon'],
  getRandomWord() {
    return this.words[Math.floor(Math.random() * this.words.length)];
  },
  checkWord(word) {
    return this.words.includes(word);
  },
  playGame() {
    let word = this.getRandomWord();
    let attempts = 0;
    while (attempts < 6) {
      let guess = prompt('Enter your guess: ');
      if (this.checkWord(guess)) {
        if (guess === word) {
          console.log('Congratulations! You guessed the word: ' + word);
          break;
        } else {
          console.log('That word is in the list, but it\'s not the word. Try again!');
        }
      } else {
        console.log('That word is not in the list. Try again!');
      }
      attempts++;
    }
  }
}
```



```
btn.addEventListener('click', () => {  
    const key = btn.dataset.key;  
    if (key === 'ENTER') this.submitGuess();  
})
```



```
const rowIdx = this.guesses.length - 1;
this.revealRow(rowIdx);
if (this.currentGuess === this.target) {
  this.won = true;
```



```
keyBtn.style.background = colors[result[i]];ð
```

```
} else if (result[i] === 'present' && current !== colors.correct) {ð
```


}}, gun: { name:'g^ah°', icon:'Ø=Ÿ+', items:['4A1 z•Qûkeg^a', 'K416 z•Qûkeg^a', 'AKM z•Qûkeg^a', 'AK-12'z•Qûkeg^a']


```
    this.results = {};  
    this.intervals = {};  
    this.render();  
  },  
  photoFor(slotKey, itemName) {  
    const items = this.slots[slotKey];
```



```
    this.intervals[k] = setInterval(() => {      const items = this.slots[k].items;      const idx = Math.floor(Math.random() * i
```



```
destroy() {    this.keys.forEach(k => clearInterval(this.intervals[k]));  };  // == Sudoku (js/games/sudoku.js) == /** * Sudoku (e
```



```
    b[r][c] = n;
```

```
    if (fill(b)) return true;
```

```
    b[r][c] = 0;
```

```
    }
```

```
    }
```



```
<button class="btn small" id="hintBtn" style="background:var(--surface2)">cBy:</button>
```

```
<select id="diffSelect" style="b
```



```
cell.style.outline = '2px solid var(--accent)'; }, fillNumber(n) { const { r, c } = this.selected; if (this.given[r][c]) r
```



```
    },    updateStats() {    if (this.statsCb) {    const filled = this.board.flat().filter(v => v !== 0).length;    this.statsCb({ ']
```


this.statsCb = statsCb; this.score = 0; this.running = true; this.started = false; this.frameCount = 0; this.pipes =

Puzzle Games Collection V3.2

// Move pipes

for (const p of this.pipes) p.x -= 2.5;

// Remove off-screen pipes

this.pipes = this.pipes.filter(p => p.x > 0);


```
c.arc(this.bird.x + 5, this.bird.y - 4, 2, 0, Math.PI * 2);    c.fill();    if (!this.started) {        c.fillStyle = '#fff';        c
```


* nñRL4ôE8y1000345900p0c0ñ000500712ac0000e0w0x-6 canvas: null, ctx: null, width: 360, height: 550, basket: { x: 0, w: 80, h: 24


```
});    this.canvas.addEventListener('touchstart', (e) => {    e.preventDefault();    const rect = this.canvas.getBoundingClient
```


Puzzle Games Collection V3.2

```
// Move items    for (const item of this.items) item.y += item.vy;    // Check catch/miss    this.items = this.items.filter(item =>
```



```
updateStats() {
  if (this.statsCb) {
    this.statsCb({ '_R ': this.score, 'u T': this.lives}})},
  gameOver() {
    this.runnin
```



```
this.tiles = Array.from({ length: this.size * this.size - 1 }, (_, i) => i + 1);    this.tiles.push(0); // 0 = empty    this.emptyI
```



```
    if (this.checkWin()) {
        this.showWin();
    },
    refreshDOM() {
        const tileEls = this.gridEl.querySelectorAll('.puzzle-tile
```


* <°_U€Ü• ^s~ß<iŷ wins< losses< draws< newGame = { board: [], player: 'X', ai: '0', turn: 'X', wins: 0, losses: 0, draws: 0, contain


```
    }    if (this.board.every(c => c !== '')) {        this.draws++;        this.updateStats();        this.endGame('^s\@ÿ ');return;    }
```



```
for (let i = 0; i < 9; i++) {  
    if (this.board[i] === '') {  
        this.board[i] = '0';  
        best = Math.max(best, thi
```



```
this.sequence.push(Math.floor(Math.random() * 4));
document.getElementById('simonStatus').textContent = 'x°00~z^•.this$UpdateSta
```



```
if (this.playerIdx >= this.sequence.length) {ð      this.score++;ð      document.getElementById('simonStatus').textContent = 'kcxnř N
```



```
    this.render();    this.updateStats(); },    render() {    this.container.innerHTML = `    <div style="text-align:center">
```



```
const delay = 1000 + Math.random() * 3000;    clearTimeout(this.timeout);    this.timeout = setTimeout(() => {        this.state = 'r
```



```
},    destroy() {      clearTimeout(this.timeout);  }); // === Number Guess (js/games/numberguess.js) === /** * Number Guessing (epfW@a
```



```
</div>    `;    const input = document.getElementById('guessInput');    document.getElementById('guessBtn').addEventListener('cl
```



```
input.value = '';
input.focus(); },
updateStats() {
  if (this.statsCb) {
    this.statsCb({ 's mKk!ep': this.attempts}})},
```



```
init(container, statsCb) {      this.container = container;      this.statsCb = statsCb;      this.score = 0;      this.streak = 0;      this.b
```



```
    this.displayColor = this.colorMap[pick];    }    const display = document.getElementById('colorDisplay');    if (display) {
```


<h3>eö•ôR0ÿ </h3>_R : \${this.score} | g 0s•pQû: \${this.bestStreak} <button class="btn" id="replayBtn">Q•geN k!</button> this.cont


```
⌀* === Header === */⌀app-header {⌀ display: flex;⌀ align-items: center;⌀ justify-content: space-between;⌀ padding: 16px 24px;⌀ background-color: #f0f0f0;⌀ border-bottom: 1px solid #ccc;⌀ margin-bottom: 10px;⌀}
```



```
␣* === 2048 === */␣game-2048 { width: 100%; max-width: 400px; }␣board-2048 {␣ display: grid;␣ grid-template-columns: repeat(4, 1fr);␣ g
```


display: grid;␣grid-template-columns: repeat(4, 70px);␣grid-template-rows: repeat(4, 70px);␣gap: 4px;␣background: var(--surface2)

inset: 0; display: flex; flex-direction: column; align-items: center; justify-content: center; background: rgba(0,0,0,0.82); border: 1px solid black; padding: 10px; text-align: center; font-family: sans-serif; font-size: 1.2em; color: white; margin: 0 auto; width: 80%; height: 100px;">

.tile-8 { background: #f2b179; color: #f9f6f2; }@tile-16 { background: #f59563; color: #f9f6f2; }@tile-32 { background: #f67c5f;

flex-wrap: wrap;@tet-btn {@ flex: 1;@ min-width: 48px;@ padding: 12px 8px !important;@ font-size: 18px !important;@ touch-action: ma

